CashMate Backend Architecture Guide

Complete Technical Documentation & File Breakdown

Executive Summary: CashMate uses a local, type-safe SQLite database powered by **Drift** (formerly Moor) and **drift_flutter**, wired directly into Flutter's UI layer via **Riverpod**. The architecture is fully offline-first, reactive (live streaming auto-updating UI), and features a hybrid schema: static structured ledger transactions combined with a flexible **Entity-Attribute-Value (EAV)** engine for user-defined custom categories and fields.

1. High-Level Architectural Flow

The backend follows clean separation of concerns across three main tiers:

- **Presentation / UI Layer (Flutter):** Widgets watch reactive Riverpod providers. Any modification immediately publishes fresh data across the entire widget tree.
- **Service / Accessor Tier (DAOs & Riverpod):** TransactionDao and CategoryDao isolate query logic and data transformations from UI widgets.
- **Core Persistence Tier (Drift & SQLite):** AppDatabase manages SQLite connection lifecycles, schema migrations, and type-safe generated companion classes.

Tier	Component / Files	Role & Responsibility
UI Consumer	lib/main.dart UI Pages / Widgets	Reads providers (databaseProvider, DAOs), listens to live streams, triggers mutations on button clicks.
Data Access	transaction_dao.dart category_dao.dart	Encapsulates queries (CRUD), multi-table joins, and provides specialized reactive streams (e.g. watchExpenses()).
Database Core	app_database.dart app_database.g.dart	Initializes SQLite database file cashmate_db, schema versioning (v1 to v2 migrations), and table registration.
Schema Definitions	lib/core/database/tables/*.dart	Declarative table schemas for transactions, categories, dynamic fields, and entry values.
Utilities	utils/value_parser.dart	Serializes / deserializes dynamic values (numbers, dates, booleans) stored in text columns.

2. Detailed Backend File Catalog

Here is the exact purpose, internals, and usage of every file residing in your backend directory:

A. lib/core/database/app_database.dart

Role: The central SQLite database coordinator.
• **Connection:** Uses driftDatabase(name: 'cashmate_db') from drift_flutter to create and manage the native SQLite database file on device storage.
• **Registry:** Annotated with @DriftDatabase(tables: [...], daos: [...]), notifying Drift code generation of all 5 tables and the 2 DAOs.
• **Schema Migration:** Maintains schemaVersion = 2. The migration getter has an onCreate block that builds all tables on fresh install, and an onUpgrade block (v1 to v2) that conditionally provisions the category tables without wiping existing v1 transaction history.

B. lib/core/database/category_dao.dart

Role: Data Access Object (DAO) for the custom category and dynamic field system.
• **CategoryFull Model:** Declares the strongly typed data class CategoryFull combining a category, its dynamic fields, entry instances, and specific field values.
• **CRUD Methods:** insertCategory, watchAllCategories(), insertField, insertEntry, insertEntryValue.
• **Complex Join:** Implements getCategoryFull(int categoryId) which executes a 4-table inner join across categories, fields, entries, and entry values, mapping the raw SQL result into CategoryFull objects.

C. lib/core/database/transaction_dao.dart

Role: Data Access Object (DAO) for financial transaction ledger records.
• **Methods:** insertTransaction, watchAllTransactions() (live descending stream by date), and watchExpenses() (pre-filtered stream for expense-only queries).
• **Mixin:** Employs _$TransactionDaoMixin generated by Drift to directly access the transactions table.

D. lib/core/database/tables/*.dart (Table Schema Definitions)

The tables/ directory defines the schema structure using Drift's declarative DSL:

File	Table Name	Columns & Types	Purpose
transactions.dart	Transactions	id (Int, PK, AutoInc) title (Text, 1-100 chars) amount (Real/Double) date (DateTime) category (Text) isExpense (Bool)	Core ledger table tracking income & expense records.
categories.dart	Categories	id (Int, PK, AutoInc) name (Text, 1-50 chars) type (Text: 'expense'/'income') icon (Text, nullable)	User or default categories grouping custom entries.
category_fields.dart	CategoryFields	id (Int, PK, AutoInc) categoryId (Int, FK -> Categories.id) fieldName (Text) fieldType (Text: 'number','date','boolean','text') sortOrder (Int, default 0)	Defines custom input fields per category (e.g. 'Mileage', 'Merchant', 'Warranty Date').
category_entries.dart	CategoryEntries	id (Int, PK, AutoInc) categoryId (Int, FK -> Categories.id) createdAt (DateTime)	An individual logged record under a category.
category_entry_values.dart	CategoryEntryValues	id (Int, PK, AutoInc) entryId (Int, FK -> CategoryEntries.id) fieldId (Int, FK -> CategoryFields.id) value (Text, nullable)	The stored value for each custom field for a given entry instance.

E. lib/core/database/utils/value_parser.dart

Role: Bridge between dynamic SQLite text columns and Dart runtime types.

• **ValueParser.serialize(fieldType, value):** Converts typed Dart values (num, DateTime, bool) into strings suitable for storage.

• **ValueParser.deserialize(fieldType, value):** Parses stored strings back into strong Dart types (e.g., parses '1' back to true, ISO strings into DateTime, strings into double).

F. Generated Files (*.g.dart)

• **app_database.g.dart:** Automatically synthesized by drift_dev. Contains SQLite table definitions ($TransactionsTable, $CategoriesTable, etc.), row data classes (Transaction, Category), companions for insert/update operations, and migration helpers.

• **category_dao.g.dart & transaction_dao.g.dart:** Contain generated accessor mixins (_$CategoryDaoMixin, _$TransactionDaoMixin) giving DAOs direct type-safe references to their attached database tables.

G. lib/main.dart (Dependency Injection Wiring)

Role: Exposes database instances to Flutter via Riverpod.

• databaseProvider: Lazily instantiates AppDatabase and safely closes it via ref.onDispose(() => db.close()).

• transactionDaoProvider & categoryDaoProvider: Inject the database into the respective DAOs.

• ProviderScope: Wraps the root widget MyApp, making all providers accessible to any UI screen via ref.watch.

3. Understanding the EAV (Entity-Attribute-Value) Strategy

Most budgeting apps only have fixed columns (amount, date, category). CashMate goes beyond this by supporting **custom category fields** without requiring SQLite schema changes every time a user creates a new field. This is implemented via the classic **Entity-Attribute-Value (EAV)** pattern:

1. **Entity Definition:** Categories (e.g. 'Car Maintenance')
2. **Attribute (Custom Field Definition):** CategoryFields (e.g. 'Odometer Reading' with type 'number')
3. **Entry Instance:** CategoryEntries (an event logged on October 13, 2026)
4. **Stored Value:** CategoryEntryValues (links the entry and field with the value '124,500')

Benefit: Users can define arbitrary fields (e.g. tags, dates, ratings, numbers) completely dynamically!

4. Quick Reference: How to Consume the Backend in UI

Here are direct examples of how to connect your new Flutter screens to the backend:

```
// 1. Reading live transactions in a widget:
class TransactionListScreen extends ConsumerWidget {
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final dao = ref.watch(transactionDaoProvider);
    return StreamBuilder<List<Transaction>>(
      stream: dao.watchAllTransactions(),
      builder: (context, snapshot) {
        final items = snapshot.data ?? [];
        return ListView.builder(
          itemCount: items.length,
          itemBuilder: (ctx, i) => ListTile(title: Text(items[i].title)),
        );
      },
    );
  }
}

// 2. Inserting a new transaction:
await ref.read(transactionDaoProvider).insertTransaction(
  title: 'Groceries', amount: 45.50, date: DateTime.now(), category: 'Food', isExpense: true,
);
```